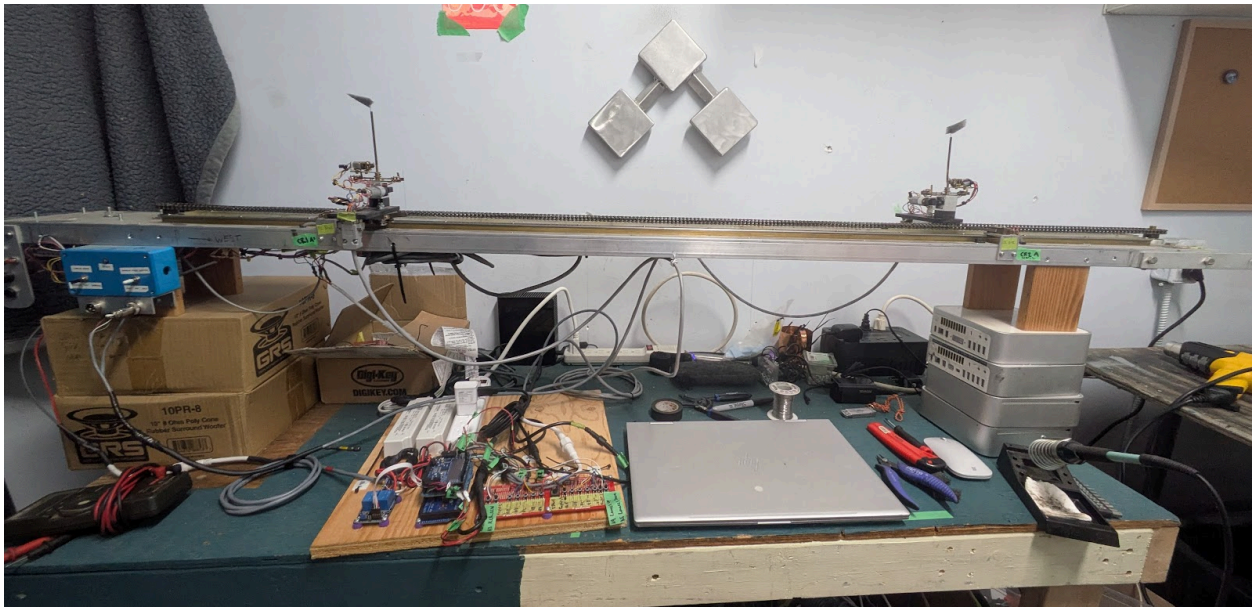
Joust Operations Manual



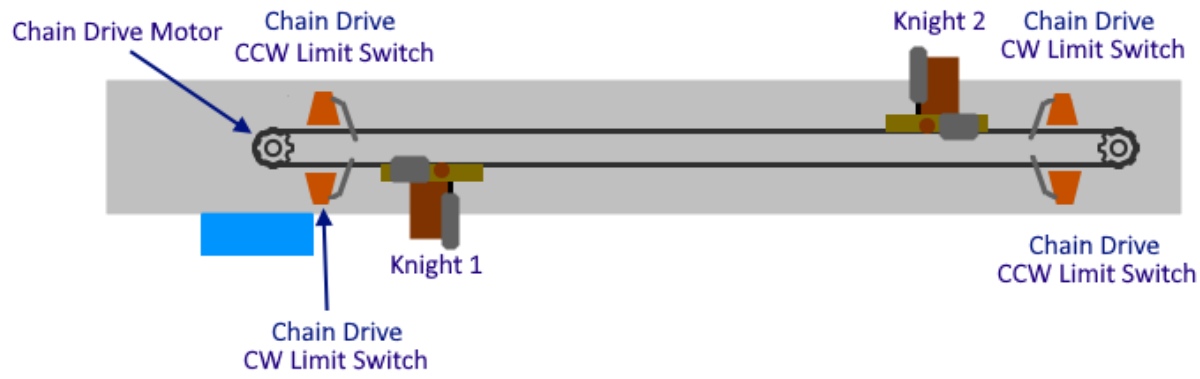


Prepared by: Nick Greeno - nickgreeno@gmail.com - (902) - 830 - 9883 or (782) 777-8822
Prepared on: 2025-03-14
Last edited: 2025-04-02

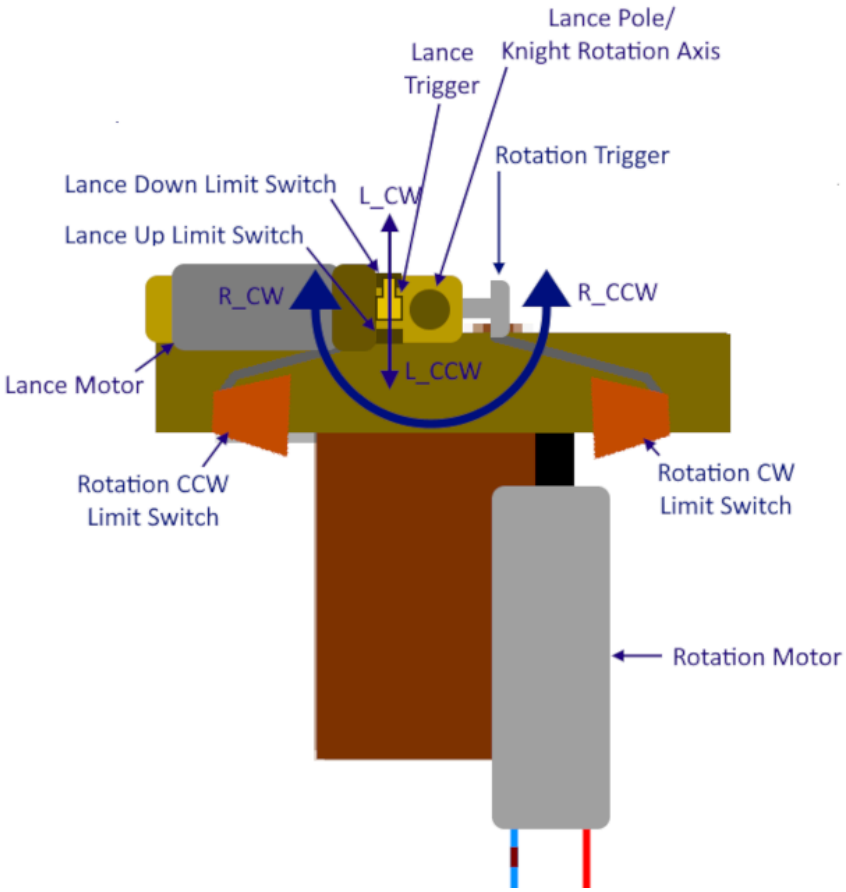
System Overview

Mechanical Systems

Main Chassis

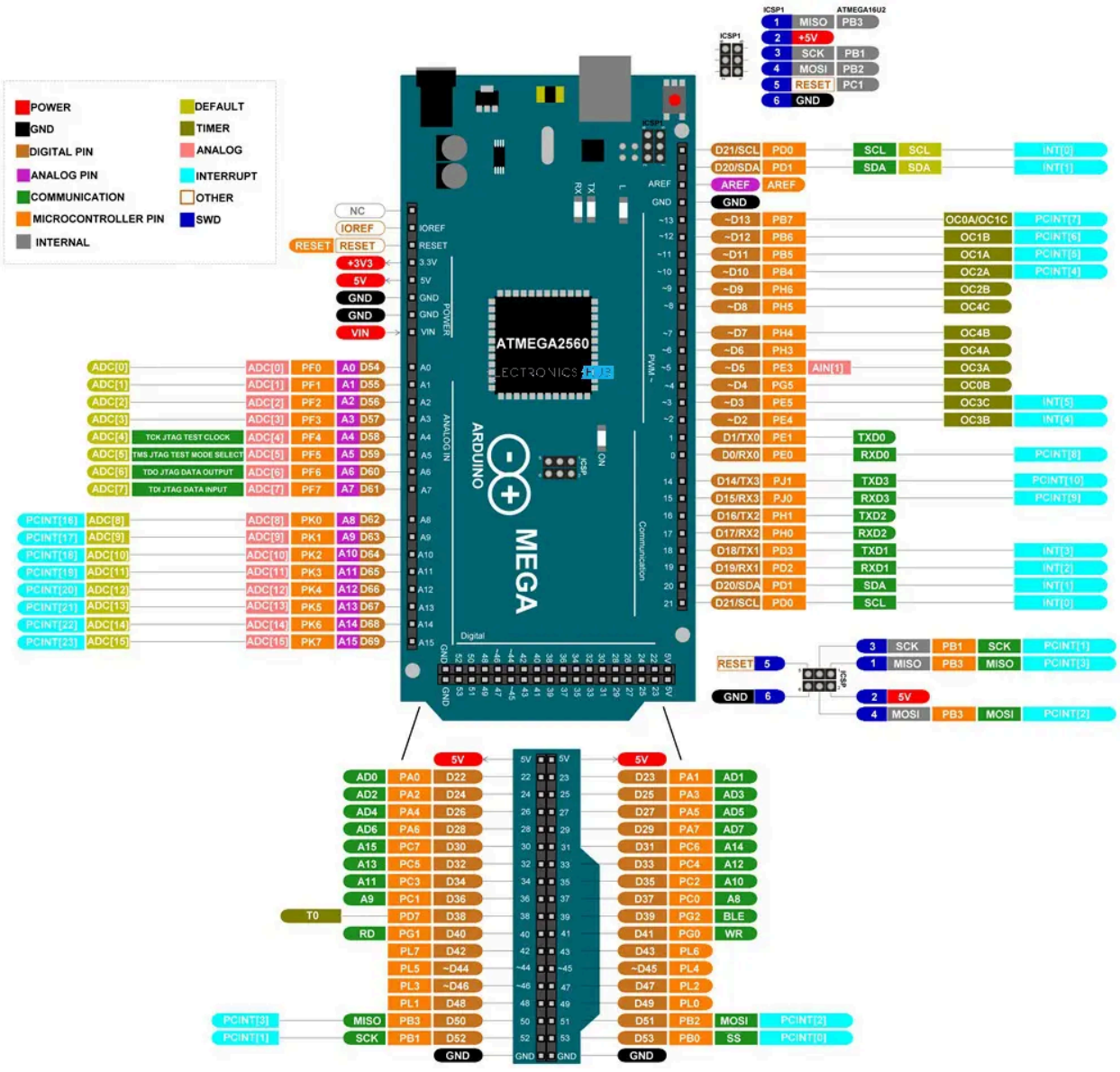


Knight Sled

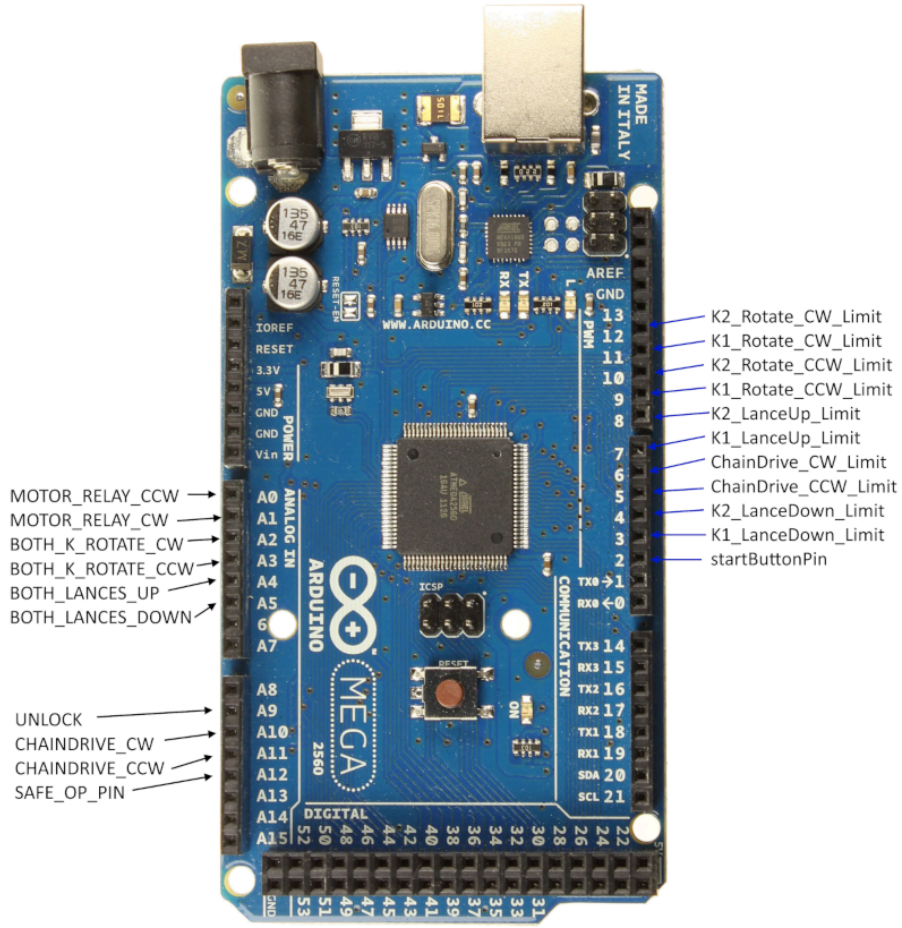


Arduino Pinout

Generic



Actual



Description	Arduino Pin
startButton	2
K1_LanceDown_limit	3
K2_LanceDown_limit	4
ChainDrive_CCW_Limit	5
ChainDrive_CW_Limit	6
K1_LanceUp_limit	7
K2_LanceUp_limit	8
K1_Rotate_CCW_Limit	9
K2_Rotate_CCW_Limit	10
K1_Rotate_CW_Limit	11
K2_Rotate_CW_Limit	12
MOTOR_RELAY_CCW	A0
MOTOR_RELAY_CW	A1
BOTH_K_ROTATE_CW	A2
BOTH_K_ROTATE_CCW	A3
BOTH_LANCES_UP	A4
BOTH_LANCES_DOWN	A5
UNLOCK	A9
CHAINDRIVE_CW	A10
CHAINDRIVE_CCW	A11
SAFE_OP_PIN	A12

Program Operation

Safe Operation Mode

When Joust is in Safe Operation Mode it will perform a series of checks to see if it is safe to operate. If these checks are passed the program will bypass the keyed lock and manual modes, perform an automatic reset, and proceed to normal operation.

To engage safe operation mode, connect a jumper from any GND pin to the A12 pin (Analog 12).

Manual mode is still accessible while the Safe Operation jumper is installed. To enter manual mode, hold down the [START] button during the bootup process. This will initiate the manual

control mode. In the unlikely event that Joust restarts and a patron is holding the start button the program will enter manual operation however, they will only have access to the [START] button so they will not be able to control the machine. Should they press the [START] button to exit manual mode the program will again check to see if it is in a recoverable state and try to auto-reset. If it is not safe to reset the program will hang, waiting for an operator to access the control panel and reset the machine to a safe position or decommission for repairs.

With the Safe Operation jumper removed, the program will progress through the phases listed below: Boot, Manual, Reset, and Normal Operation.

On Boot

Joust starts in locked state so long as the Safe Operation jumper is not installed. No user input will be recognized in this state, no motors can engage. In the event of a critical failure the program will timeout and reset after 2 seconds, returning to this state.

When locked, the start button will turn on for 3 seconds, then turn off for 3 seconds.

Joust is unlocked by using the key. Insert key and turn to unlock. This will enter manual mode. When in manual mode, the start button will turn on for one second, then turn off for one second. Once in manual mode the key can return to the locked position and be removed.

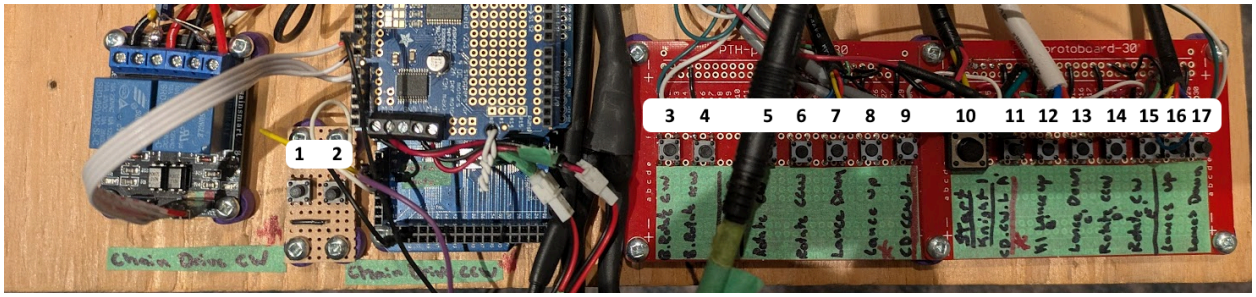
Manual Operation

The user has full control over all motors and can simulate limit switch triggers. Limit switches have been implemented such that the motors will not overdrive.

WARNING: While in manual mode nights can rotate while in the middle of the track, potentially causing a collision with the fence.

To exit manual mode, press the start button on the PCB or the main start button on the display.

Control Panel



1. Chain Drive CCW - Engages the chain drive motor in the counter clockwise direction. Knight 1 (closest to user) moves the to left, Knight 2 (furthest from user) moves to the right. Pressing button [11] will trigger the chain drive CCW limit switch to stop the chain drive motor.
2. Chain Drive CW - Engages the chain drive motor in the clockwise direction. Knight 1 (closest to user) moves the to right, Knight 2 (furthest from user) moves to the left. Pressing button [9] will trigger the chain drive CW limit switch to stop the chain drive motor.
3. Both Knights Rotate CW - Both knights will rotate in CW direction. Pressing button [15] will stop Knight 1. Pressing button [5] will stop Knight 2.
4. Both Knights Rotate CCW - Both knights will rotate in CCW direction. Pressing button [14] will stop Knight 1. Pressing button [6] will stop Knight 2.
5. Knight 2 - CW Rotation Limit Switch - Triggers the end of Knight 2 CW rotation.
6. Knight 2 - CCW Rotation Limit Switch - Triggers the end of Knight 2 CCW rotation.
7. Knight 2 - Lance Down Limit Switch - Triggers the end of Knight 2 lance down movement.
8. Knight 2 - Lance Up Limit Switch - Triggers the end of Knight 2 lance up movement.
9. Chain Drive CW Limit Switch - Triggers the end of the chain drive CW rotation.
10. Start Button - Emulates the Start button when installed in display case. Exits Manual mode and initiates reset mode.
11. Chain Drive CCW Limit Switch - Triggers the end of the chain drive CCW rotation.
12. Knight 1 - Lance Up Limit Switch - Triggers the end of Knight 2 lance up movement.
13. Knight 1 - Lance Down Limit Switch - Triggers the end of Knight 1 lance down movement.
14. Knight 1 - CCW Rotation Limit Switch - Triggers the end of Knight 1 CCW rotation.
15. Knight 1 - CW Rotation Limit Switch - Triggers the end of Knight 1 CW rotation.
16. Both Lance Up - Both knights will raise their lances. Pressing button [12] will stop Knight 1. Pressing button [8] will stop Knight 2.
17. Both Lances Down - Both knights will lower their lances. Pressing button [13] will stop Knight 1. Pressing button [7] will stop Knight 2.

Reset Mode

Once manual mode is exited the program will go into reset mode. The machine will run a series of checks to determine where the knights are located, their rotation, and their lance position. In most cases the reset will be automatic, however, there is one condition that would require user input, this is unlikely to ever happen during normal operations. This condition could be achieved using manual mode.

Once the reset has been completed the program will enter the Joust routine.

Fully Automatic Reset Conditions

If either the chain drive CW or CCW limit switch is closed, the lances with raise and knights will rotate to face the appropriate direction.

If either of the knights are fully rotated CW or CCW the other knight will be rotated to match orientation and the chain drive will engage in the appropriate direction to return the knights “home”.

Semi-Automatic Reset Condition

If both chain drive limit switch are open and all 4 rotation limit switches are open the program will wait for the user to select knights rotate CW or CCW (same buttons as in manual mode) then proceed to return the knights to the appropriate “home” location based on their orientation.

The start button will turn on for one second then turn off for one second while waiting for input.

Joust - Normal Operation

When ready to joust, the program will turn on the start button light. When pressed, the start button will turn off until the current joust is completed.

Stages of joust

Salute: The knights will bow their lances.

Charge: The knights will lower their lances then charge towards each other.

Impact: Where the knights would impact each other.

Pass: The phase after impact.

Turn: The knights will raise their lances then turn.

Reset: A winner is declared (at random). This knight will bow his lance. The program is then ready to run again.

Safety Features

Limit Switches

Any time a call to engage a motor is made the program checks to ensure that the limit switch is open. If it is closed, that motor will not engage. This is done on a per-motor basis, such that if

both knights rotate CCW is called but one is already rotated CCW, the other will still rotate appropriately.

Watchdog Timer

A watchdog timer of 2 seconds has been implemented. If the program gets stuck for any reason, the Watchdog timer will trigger and reset the program and will not be able to re-engage until unlocked using the key.

Physical Key

A key slot has been implemented to unlock the machine when first powered up or when the watchdog timer has been triggered.

The key acts as a button. Once the key is turned to the unlock position the program will enter manual mode. Once manual mode is engaged the key should be return to the locked position and removed.

If the key is removed while in the locked position this will not affect the operation of the program. If the watchdog timer is triggered the key will have to be used to lock and then unlock the machine as it is looking for a change in state of the “button”.

Lance Redundancy Check

During testing it was discovered that the lance up/down limit switches do not always register when engaged. This would cause the program to stall. A timer has been implemented to turn off the lance motors after a specified time and moth to the next phase of the program. During testing, this happened approximately 4/150 runs with no interference to the overall operation of the program. This timer does not trigger the watchdog reset.

UPS Backup Power

A UPS backup is recommended but not critical. A UPS would ensure that during a power outage or brown out the machine continues operation as normal. This is not necessary as the machine can automatically reset from any power loss condition.

Arduino Code

```
#include <Adafruit_MotorShield.h>
#include <ezButton.h>
#include<avr/wdt.h>

#define DEBUG           0    //  Set to false (0) for deployment, true (1) for debugging
#define LOCK_BYPASS     0    //  Set to false (0) to bypass key unlock
//#define SAFE_OP         0    //  When true (1) bypasses lock and manual mode if safe to start. Hold START button to enter manual mode

//relay control pin on analog pins because all the digital ones are used
#define MOTOR_RELAY_CCW        A0
#define MOTOR_RELAY_CW         A1
#define BOTH_K_ROTATE_CW       A2
#define BOTH_K_ROTATE_CCW      A3
#define BOTH_LANCES_UP         A4
#define BOTH_LANCES_DOWN       A5
#define UNLOCK                 A9
#define CHAINDRIVE_CW          A10
#define CHAINDRIVE_CCW         A11
#define SAFE_OP_PIN            A12

//Simple Delays
#define joustDelay          100
#define joustTurnDelay      500
#define watchdogTimeoutL    3300    //Lance time = 3000
#define watchdogTimeoutCD   9600    //Chain Drive time Avg = 9300
#define watchdogTimeoutKR   31000    //Rotation time = 30500

//Define rotate speed for lance and rotate motors
#define lanceSpeed          50
#define debounceTimeSwitch   50
#define debounceTimeButton   50
#define rotateSpeed          150

/******* Arduino Pin Configuration ************************/
// Pin 0: not to be used >>> internal Tx
// Pin 1: not to be used >>> internal Rx
// See documentation: [Add Website]

int startButtonPin       = 2;      //  [START]
int K1_LanceDown_Limit   = 3;      //  [B']
int K2_LanceDown_Limit   = 4;      //  [B ]
```

```
int ChainDrive_CCW_Limit  = 5;      //  [A ]
int ChainDrive_CW_Limit   = 6;      //  [A']
int K1_LanceUp_Limit      = 7;      //  [C']
int K2_LanceUp_Limit      = 8;      //  [C ]
int K1_Rotate_CCW_Limit   = 9;      //  [D']
int K2_Rotate_CCW_Limit  = 10;      //  [D ]
int K1_Rotate_CW_Limit   = 11;      //  [E']
int K2_Rotate_CW_Limit   = 12;      //  [E ]

//Indicator LED
int startButtonLED       = 13;      //  Onboard LED
bool stateLED            = false;

/*
The resistor is essential for safe operation as it limits the current flowing through the LED,
preventing damage to both the LED and the Arduino's output pin. You can choose the resistor
value
based on the desired current using Ohm's Law (V = IR) where V is the voltage of your board (5V
or 3.3V)
minus the forward voltage for the LED you are using (typical for red would be 1.8 to 2.2 volts).
In this case, using a 220-ohm resistor with an Arduino UNO R3 (a 5V board) limits the current to
a safe level for both the LED and the Arduino pin. Adjusting the resistor value allows you to
control
the LED's brightness while ensuring safe operation. For 5V boards you can expect the LED to
be visible
to a resistor value of up to 1K Ohm.
*/

//timing
int startMillis;
int currentMillis;
int millisPeriod = 1000;
int lanceMillisPeriod = 3000;
int unlockPeriod = 3000;

//Counters
int joustRuns = 0;
int winsK1    = 0;
int winsK2    = 0;
int lanceFail = 0;

//Program Structure
#if DEBUG
bool firstLoop    = true;
```

```
bool manualLoop   = true;
bool resetLoop    = false;
#else
bool firstLoop    = true;
bool manualLoop   = true;
bool resetLoop    = true;
#endif

#if LOCK_BYPASS
bool locked       = false;
#else
bool locked       = true;
#endif

bool safeStart    = false;
bool joustLoop    = true;
bool userInput    = false;

bool K1_Check     = false;
bool K2_Check     = false;
bool K1_Check_2   = false;
bool K2_Check_2   = false;
bool checkConfirm = false;

bool rotate = true;

int winner;

/*********************************
****** W A R N I N G *************
*********************************
```

ANALOG PIN 4 CANNOT BE CONNECTED TO GROUND AT BOOT OR ELSE MOTOR SHIELD IS NOT RECOGNIZED!!!!!

From https://learn.adafruit.com/adafruit-shield-compatibility?view=all

Do not use Analog pins 4 or 5 with your project in combination with this shield, even on Arduino boards that use different pins for I²C communication (Leonardo, Mega, Due). For compatibility with
“classic” (pre-R3) Arduino boards, the shield connects SDA and SCL to Analog pins 4 and 5, respectively;
communication and analog readings would mutually interfere. Leonardo users should likewise avoid Digital

```
pins 2 and 3 when using I²C.

**********************************/

// Create the motor shield object with the default I2C address
Adafruit_MotorShield AFMS = Adafruit_MotorShield();
// Or, create it with a different I2C address (say for stacking)
// Adafruit_MotorShield AFMS = Adafruit_MotorShield(0x61);

// Select which 'port' M1, M2, M3 or M4. In this case, M1
Adafruit_DCMotor *M1_K1_LANCE   = AFMS.getMotor(1);
Adafruit_DCMotor *M2_K1_ROTATE  = AFMS.getMotor(2);
Adafruit_DCMotor *M3_K2_LANCE   = AFMS.getMotor(3);
Adafruit_DCMotor *M4_K2_ROTATE  = AFMS.getMotor(4);

//Limit Switches
ezButton K1_LanceUp_Switch      (K1_LanceUp_Limit,        INPUT_PULLUP);
ezButton K2_LanceUp_Switch      (K2_LanceUp_Limit,        INPUT_PULLUP);
ezButton ChainDrive_CCW_Switch  (ChainDrive_CCW_Limit,    INPUT_PULLUP);
ezButton ChainDrive_CW_Switch   (ChainDrive_CW_Limit,     INPUT_PULLUP);
ezButton K1_LanceDown_Switch    (K1_LanceDown_Limit,      INPUT_PULLUP);
ezButton K2_LanceDown_Switch    (K2_LanceDown_Limit,      INPUT_PULLUP);
ezButton K1_Rotate_CCW_Switch   (K1_Rotate_CCW_Limit,     INPUT_PULLUP);
ezButton K2_Rotate_CCW_Switch   (K2_Rotate_CCW_Limit,     INPUT_PULLUP);
ezButton K1_Rotate_CW_Switch    (K1_Rotate_CW_Limit,      INPUT_PULLUP);
ezButton K2_Rotate_CW_Switch    (K2_Rotate_CW_Limit,      INPUT_PULLUP);

//User Input Buttons
ezButton startButton                (startButtonPin,    INPUT_PULLUP);
ezButton Both_Knights_Rotate_CW_Button  (BOTH_K_ROTATE_CW,  INPUT_PULLUP);
ezButton Both_Knights_Rotate_CCW_Button (BOTH_K_ROTATE_CCW, INPUT_PULLUP);
ezButton Both_LancesDown_Button         (BOTH_LANCES_DOWN,  INPUT_PULLUP);
ezButton Both_LancesUp_Button           (BOTH_LANCES_UP, INPUT_PULLUP);
ezButton ChainDrive_CW                  (CHAINDRIVE_CW, INPUT_PULLUP);
ezButton ChainDrive_CCW                 (CHAINDRIVE_CCW, INPUT_PULLUP);
ezButton Unlock                         (UNLOCK, INPUT_PULLUP);

void setup(){
  Serial.begin(9600);

  if (!AFMS.begin()) {  // create with the default frequency 1.6KHz
    // if (!AFMS.begin(1000)) {  // OR with a different frequency, say 1KHz
    Serial.print("Could not find Motor Shield. Check wiring.");
    while (1);
```

```
}

//setup switch debouncing
K1_LanceUp_Switch.setDebounceTime(debounceTimeSwitch);
K2_LanceUp_Switch.setDebounceTime(debounceTimeSwitch);
ChainDrive_CCW_Switch.setDebounceTime(debounceTimeSwitch);
ChainDrive_CW_Switch.setDebounceTime(debounceTimeSwitch);
K1_LanceDown_Switch.setDebounceTime(debounceTimeSwitch);
K2_LanceDown_Switch.setDebounceTime(debounceTimeSwitch);
K1_Rotate_CCW_Switch.setDebounceTime(debounceTimeSwitch);
K2_Rotate_CCW_Switch.setDebounceTime(debounceTimeSwitch);
K1_Rotate_CW_Switch.setDebounceTime(debounceTimeSwitch);
K2_Rotate_CW_Switch.setDebounceTime(debounceTimeSwitch);

//User Input Buttons
startButton.setDebounceTime(debounceTimeButton);
Both_Knights_Rotate_CW_Button.setDebounceTime(debounceTimeButton);
Both_Knights_Rotate_CCW_Button.setDebounceTime(debounceTimeButton);
Both_LancesDown_Button.setDebounceTime(debounceTimeButton);
Both_LancesUp_Button.setDebounceTime(debounceTimeButton);
ChainDrive_CW.setDebounceTime(debounceTimeButton);
ChainDrive_CCW.setDebounceTime(debounceTimeButton);
Unlock.setDebounceTime(debounceTimeButton);

//setup relay control pins on A0 and A1
pinMode(MOTOR_RELAY_CW, OUTPUT);
pinMode(MOTOR_RELAY_CCW, OUTPUT);

//Safe_Op Hardware Init
pinMode(SAFE_OP_PIN, INPUT_PULLUP);

//Start LED
pinMode(LED_BUILTIN, OUTPUT); //startButtonLED

//make sure relays are both in OFF state HIGH == OFF, LOW == ON
digitalWrite(MOTOR_RELAY_CW, HIGH);
digitalWrite(MOTOR_RELAY_CCW, HIGH);

// Initialize Motors
M1_K1_LANCE->setSpeed(lanceSpeed);
M1_K1_LANCE->run(FORWARD);
M1_K1_LANCE->run(RELEASE);

M2_K1_ROTATE->setSpeed(rotateSpeed);
```

```
  M2_K1_ROTATE->run(FORWARD);
  M2_K1_ROTATE->run(RELEASE);

  M3_K2_LANCE->setSpeed(lanceSpeed);
  M3_K2_LANCE->run(FORWARD);
  M3_K2_LANCE->run(RELEASE);

  M4_K2_ROTATE->setSpeed(rotateSpeed);
  M4_K2_ROTATE->run(FORWARD);
  M4_K2_ROTATE->run(RELEASE);

  startMillis = millis();

  wdt_disable();
  delay(3000);
  wdt_enable(WDTO_1S);

  if (!digitalRead(SAFE_OP_PIN)){
    safeStart = true;
    Serial.print("Safe Start Mode\n");
  }
}

void loop(){
  wdt_reset();

  if (firstLoop){
    Serial.print("Joust: Online!\n");
    Serial.print("Safe Start: "); Serial.print(safeStart); Serial.print("\n");


    if (safeStart){
      if (check_Safe_Operation()){
        locked = false;
        manualLoop = false;
      }

      startButton.loop();
      if (startButton.isPressed() || !check_startButton()){
        Serial.print("Enter Manual Mode\n");
        startButton.loop();
        manualLoop = true;
        locked = false;
      }
```

```
  }

  while (locked){
    wdt_reset();

    currentMillis = millis();
    if (currentMillis - startMillis >= unlockPeriod){
      Serial.print("Waiting for Unlock\n");
      digitalWrite(LED_BUILTIN, stateLED);
      stateLED = !stateLED;
      startMillis = currentMillis;
    }

    Unlock.loop();
    if (Unlock.isPressed() || !check_LOCK()){
      wdt_reset();
      Unlock.loop();
      locked = false;
    }
}

  while (manualLoop){
    wdt_reset();
    //Manual Mode
    currentMillis = millis();
    if ( currentMillis - startMillis >= millisPeriod){
      Serial.print("Manual control mode\n");
      digitalWrite(LED_BUILTIN, stateLED);
      stateLED = !stateLED;
      startMillis = currentMillis;
    }

    delay(joustDelay);
    allButtonLoops();

    //Lances Up
    if (Both_LancesUp_Button.isPressed()){
      allButtonLoops();
      Serial.print("Both Lances UP\n");
      raiseLances();
      delay(joustDelay);
    }

    //Lances Down
```

```
if (Both_LancesDown_Button.isPressed()){
  allButtonLoops();
  Serial.print("Both Lances DOWN\n");
  lowerLances();
  delay(joustDelay);
}

//Rotate Both Knights CW
if (Both_Knights_Rotate_CW_Button.isPressed()){
  allButtonLoops();
  Serial.print("Both Knights Rotate: CW\n");
  startMillis = millis();
  bothKnightsRotate_CW();
  currentMillis = millis();
  Serial.print("KR_CW time: "); Serial.print(currentMillis-startMillis); Serial.print("\n");
  delay(joustDelay);
}

//Rotate Both Knights CCW
if (Both_Knights_Rotate_CCW_Button.isPressed()){
  allButtonLoops();
  Serial.print("Both Knights Rotate: CCW\n");
  startMillis = millis();
  bothKnightsRotate_CCW();
  currentMillis = millis();
  Serial.print("KR_CCW time: "); Serial.print(currentMillis-startMillis); Serial.print("\n");
  delay(joustDelay);
}

//Chain Drive CW
if (ChainDrive_CW.isPressed()){ //Press the LIMIT you want the Knight to hit
  allButtonLoops();
  Serial.print("Engage Chain Drive: CW\n");
  delay(joustDelay);
  startMillis = millis();
  engage_ChainDrive_CW();
  currentMillis = millis();
  Serial.print("CD_CW time: "); Serial.print(currentMillis-startMillis); Serial.print("\n"); //9400
}

//Chain Drive CCW
if (ChainDrive_CCW.isPressed()){ //Press the LIMIT you want the Knight to hit
  allButtonLoops();
  Serial.print("Engage Chain Drive: CCW\n");
```

```
      delay(joustDelay);
      startMillis = millis();
      engage_ChainDrive_CCW();
      currentMillis = millis();
      Serial.print("CD_CCW time: "); Serial.print(currentMillis-startMillis); Serial.print("\n"); //9400
    }

    //startButton.loop();
    if(startButton.isPressed()){
      allButtonLoops();
      firstLoop = false;
      manualLoop = false;
      delay(joustDelay);
    }
  }

  if (resetLoop){
    wdt_reset();
    Serial.print("Check Starting Conditions\n");
    delay(joustDelay);

    //Raise the lances
    Serial.print("Raise Lances\n");
    raiseLances();

    delay(joustDelay);

    //If knights are "Home" rotation is safe
    if(!check_ChainDrive_CW_Limit()){
      bothKnightsRotate_CW();
    }

    if(!check_ChainDrive_CCW_Limit()){
      bothKnightsRotate_CCW();
    }

    delay(joustDelay);

    //Check Knight positions
    //Neither Knight has engaged either CW or CCW Rotation Limits
    if (((check_K1_Rotate_CW_Limit()) && (check_K1_Rotate_CCW_Limit())) &&
((check_K2_Rotate_CW_Limit()) && (check_K2_Rotate_CCW_Limit()))){
      while (!userInput){
        wdt_reset();
```

```
        allButtonLoops();

        currentMillis = millis();
        if ( currentMillis - startMillis >= millisPeriod){
          digitalWrite(LED_BUILTIN, stateLED);
          Serial.print("Waiting for User input\n");
          stateLED = !stateLED;
          startMillis = currentMillis;
          wdt_reset();
        }

        //Rotate Both Knights CW
        if (Both_Knights_Rotate_CW_Button.isPressed()){
          allButtonLoops();
          Serial.print("Both Knights Rotate: CW\n");
          bothKnightsRotate_CW();
          userInput = true;
          delay(joustDelay);
          wdt_reset();
        }

        //Rotate Both Knights CCW
        if (Both_Knights_Rotate_CCW_Button.isPressed()){
          allButtonLoops();
          Serial.print("Both Knights Rotate: CCW\n");
          bothKnightsRotate_CCW();
          userInput = true;
          delay(joustDelay);
          wdt_reset();
        }
       }
      }
      wdt_reset();
      delay(joustDelay);
      wdt_reset();

      //Check Knights CW orientation
      if (((check_K1_Rotate_CW_Limit()) && (!check_K2_Rotate_CW_Limit())) ||
((!check_K1_Rotate_CW_Limit()) && (check_K2_Rotate_CW_Limit()))){
        bothKnightsRotate_CW();
        delay(joustDelay);
      }

      //Check Knights CCW orientation
```

```
    if (((check_K1_Rotate_CCW_Limit()) && (!check_K2_Rotate_CCW_Limit())) ||
((!check_K1_Rotate_CCW_Limit()) && (check_K2_Rotate_CCW_Limit()))){
      bothKnightsRotate_CCW();
      delay(joustDelay);
    }

    //Check Chain Drive, knights should now be set so its safe to engage
    if((check_ChainDrive_CW_Limit()) || (check_ChainDrive_CCW_Limit())){
      //If knights are CW chain drive goes CW
      if((!check_K1_Rotate_CW_Limit()) && (!check_K2_Rotate_CW_Limit())){
        engage_ChainDrive_CW();
        delay(joustDelay);
      }

      //If knights are CCW Chain Drive goes CCW
      if((!check_K1_Rotate_CCW_Limit()) && (!check_K2_Rotate_CCW_Limit())){
        engage_ChainDrive_CCW();
        delay(joustDelay);
      }
    }
  }
  digitalWrite(LED_BUILTIN, HIGH);
  firstLoop = false;
 }

 if (joustLoop){
  wdt_reset();
  currentMillis = millis();
  if (currentMillis - startMillis >= millisPeriod){
    Serial.print("Ready to Joust...\n");
    startMillis = currentMillis;
  }

  startButton.loop();
  if(startButton.isPressed()){
    startButton.loop();
    digitalWrite(LED_BUILTIN, LOW);
    Serial.print("Start the Joust!\n");

    delay(joustDelay);
    wdt_reset();

    //SALUTE
    lowerLances();
```

```
    delay(joustDelay);
    wdt_reset();

    raiseLances();
    delay(joustTurnDelay);
    wdt_reset();

    //CHARGE
    lowerLances();
    delay(joustTurnDelay);
    wdt_reset();

    if (!check_ChainDrive_CW_Limit() && ((!check_K1_Rotate_CW_Limit()) ||
(!check_K2_Rotate_CW_Limit()))){
      engage_ChainDrive_CCW();
    }
    if (!check_ChainDrive_CCW_Limit() && ((!check_K1_Rotate_CCW_Limit()) ||
(!check_K2_Rotate_CCW_Limit()))){
      engage_ChainDrive_CW();
    }

    delay(joustTurnDelay);
    wdt_reset();
    //PASS

    raiseLances();
    delay(joustTurnDelay);
    wdt_reset();

    //TURN
    if (!check_ChainDrive_CCW_Limit()){ //rotate knights CCW
      bothKnightsRotate_CCW();
    }

    if (!check_ChainDrive_CW_Limit()){  //rotate knights CW
      bothKnightsRotate_CW();
    }

    delay(joustTurnDelay);
    wdt_reset();

    Serial.print("Joust Complete!\n");

    //Stretch Goal: Pick true/false at random for winner
```

```
winner = random(0,2);
Serial.print("Winner: ");
Serial.print(winner);
Serial.print("\n");

if(winner == 0){
  winsK1++;
  Serial.print("Winner: K1. Total Wins: ");
  Serial.print(winsK1);
  Serial.print("\n");
  engage_K1_LanceDown();
  delay(joustDelay);
  engage_K1_LanceUp();
}

if (winner == 1){
  winsK2++;
  Serial.print("Winner: K2. Total Wins: ");
  Serial.print(winsK2);
  Serial.print("\n");
  engage_K2_LanceDown();
  delay(joustDelay);
  engage_K2_LanceUp();
}

joustRuns++;

Serial.print("******************\n");
Serial.print("Joust runs: ");
Serial.print(joustRuns);
Serial.print("\n");

Serial.print("Wins K1: ");
Serial.print(winsK1);
Serial.print("\n");

Serial.print("Wins K2: ");
Serial.print(winsK2);
Serial.print("\n");

Serial.print("Lance Recoveries: ");
Serial.print(lanceFail);
Serial.print("\n");
Serial.print("******************\n");
```

```
      delay(joustTurnDelay);
      wdt_reset();

      digitalWrite(LED_BUILTIN, HIGH);
    }
  }
}

/***** Motor Handler Functions *****/
//Check to make sure safe to engage motor before engaging
//Chain Drive Handler Functions
void engage_ChainDrive_CW(){
  Serial.print("Chain Drive called for CW\n");
   rotate = true;
   checkConfirm = true;

  //Check Chain Drive CW Limit Switch
  if(check_ChainDrive_CW_Limit()){
    delay(joustDelay);
    Serial.print("Safe to engage Chain Drive CW\n");
    chainDrive_CW();
    startMillis = millis();

    while(checkConfirm){
      wdt_reset();
      while(rotate){
        wdt_reset();
        ChainDrive_CW_Switch.loop();
        if(ChainDrive_CW_Switch.isPressed()){
          Serial.print("Chain Drive CW Limit detected\n");
          chainDrive_STOP();
          rotate = false;
        }

        currentMillis = millis();
        if ( currentMillis - startMillis >= watchdogTimeoutCD){
          while(1){}
        }
      }

      if(!check_ChainDrive_CW_Limit()){
        Serial.print("Chain Drive CW Limit confirmed\n");
        checkConfirm = false;
```

```
      }

      else {
        rotate = true;
        checkConfirm = true;
      }
    }
  }

  else {
    Serial.print("Chain Drive is already at CW limit\n");
  }
  delay(joustDelay);
}
void engage_ChainDrive_CCW(){
  Serial.print("Chain Drive called for CCW\n");
   rotate = true;
   checkConfirm = true;

  //Check Chain Drive CW Limit Switch
  if(check_ChainDrive_CCW_Limit()){
    delay(joustDelay);
    Serial.print("Safe to engage Chain Drive CW\n");
    chainDrive_CCW();
    startMillis = millis();

    while(checkConfirm){
      wdt_reset();
      while(rotate){
        wdt_reset();
        ChainDrive_CCW_Switch.loop();

        if(ChainDrive_CCW_Switch.isPressed()){
          Serial.print("Chain Drive CCW Limit detected\n");
          chainDrive_STOP();
          rotate = false;
        }

        currentMillis = millis();
        if (currentMillis - startMillis >= watchdogTimeoutCD){
          while(1){}
        }

      }
```

```
      if(!check_ChainDrive_CCW_Limit()){
        Serial.print("Chain Drive CCW Limit confirmed\n");
        checkConfirm = false;
      }

      else {
        rotate = true;
        checkConfirm = true;
      }
    }
  }

  else {
    Serial.print("Chain Drive is already at CCW limit\n");
  }
  delay(joustDelay);
}

//Both Knights Rotate Motor Handler Functions
void bothKnightsRotate_CW(){
  //Check to make sure K1 and K2 can safely Rotate CW
  //Stop motor when limit has been detected

  Serial.print("Call for: Both Knights Rotate CW\n");
  K1_Check      = false;
  K2_Check      = false;
  K1_Check_2    = false;
  K2_Check_2    = false;
  checkConfirm  = false;

  //Check K1 and K2 Orientation
  if(check_K1_Rotate_CW_Limit()){
    K1_rotate_CW();
  }
  else {
    Serial.print("K1 already at CW Limit\n");
    K1_Check = true;
  }

  if (check_K2_Rotate_CW_Limit()){
    K2_rotate_CW();
  }
  else{
```

```
    Serial.print("K2 already at CW Limit\n");
    K2_Check = true;
   }

   if ( K1_Check && K2_Check){
    checkConfirm = true;
   }

   startMillis = millis();

   while((!K1_Check) || (!K2_Check)){
    wdt_reset();
    allButtonLoops();

    if(K1_Rotate_CW_Switch.isPressed() && (!K1_Check)){
     allButtonLoops();
     Serial.print("K1 Rotate CW Limit detected\n");
     K1_rotate_STOP();
     K1_Check = true;
    }

    if(K2_Rotate_CW_Switch.isPressed() && (!K2_Check)){
     allButtonLoops();
     Serial.print("K2 Rotate CW Limit detected\n");
     K2_rotate_STOP();
     K2_Check = true;
    }

    currentMillis = millis();
    if (currentMillis - startMillis >= watchdogTimeoutKR){
     while(1){}
    }
   }
}

void bothKnightsRotate_CCW(){
  //Check to make sure K1 and K2 can safely Rotate CCW
  //Stop motor when limit has been detected

  Serial.print("Call for: Both Knights Rotate CCW\n");
  K1_Check      = false;
  K2_Check      = false;
  K1_Check_2    = false;
  K2_Check_2    = false;
```

```
checkConfirm  = false;

//Check K1 and K2 Orientation
if(check_K1_Rotate_CCW_Limit()){
  K1_rotate_CCW();
}
if(!check_K1_Rotate_CCW_Limit()) {
  Serial.print("K1 already at CCW Limit\n");
  K1_Check = true;
}

if (check_K2_Rotate_CCW_Limit()){
  K2_rotate_CCW();
}
if (!check_K2_Rotate_CCW_Limit()){
  Serial.print("K2 already at CCW Limit\n");
  K2_Check = true;
}

if ( K1_Check && K2_Check){
  checkConfirm = true;
}

startMillis = millis();

while((!K1_Check) || (!K2_Check)){
  wdt_reset();
  allButtonLoops();

  if(K1_Rotate_CCW_Switch.isPressed() && (!K1_Check)){
    allButtonLoops();
    Serial.print("K1 Rotate CCW Limit detected\n");
    K1_rotate_STOP();
    K1_Check = true;
  }

  if(K2_Rotate_CCW_Switch.isPressed() && (!K2_Check)){
    allButtonLoops();
    Serial.print("K2 Rotate CCW Limit detected\n");
    K2_rotate_STOP();
    K2_Check = true;
  }

  currentMillis = millis();
```

```
    if (currentMillis - startMillis >= watchdogTimeoutKR){
      while(1){}
    }
  }
}

//Knight 1: Rotate Motor Handler Functions
void engage_K1_Rotate_CW(){
  //Check to make sure K1 can safely rotate CW then engage
  //Stop motor when limit has been detected

  Serial.print("K1 Rotate called for CW\n");
   K1_Check = true;
   checkConfirm = true;

  //Check K1 Rotate CW Limit Switch
  if(check_K1_Rotate_CW_Limit()){
    Serial.print("Safe to engage K1 Rotate CW\n");
    K1_rotate_CW();
    startMillis = millis();

    while(checkConfirm){
      wdt_reset();
      while(rotate){
        wdt_reset();
        K1_Rotate_CW_Switch.loop();
        if(K1_Rotate_CW_Switch.isPressed()){
          Serial.print("K1 rotate CW Limit detected\n");
          rotate = false;
        }
      }

      if(check_K1_Rotate_CW_Limit() == false){
        Serial.print("K1 rotate CW Limit confirmed\n");
        checkConfirm = false;
      }

      else {
        K1_Check = true;
        checkConfirm = true;
      }

      currentMillis = millis();
      if (currentMillis - startMillis >= watchdogTimeoutKR){
```

```
        while(1){}
      }
    }
    K1_rotate_STOP();
  }

  else {
    Serial.print("K1 Rotate is already at CW limit\n");
  }
}
void engage_K1_Rotate_CCW(){
  Serial.print("K1 Rotate called for CCW\n");
   K1_Check = true;
   checkConfirm = true;

  //Check K1 Rotate CCW Limit Switch
  if(check_K1_Rotate_CCW_Limit()){
    Serial.print("Safe to engage K1 Rotate CCW\n");
    K1_rotate_CCW();
    startMillis = millis();

    while(checkConfirm){
      wdt_reset();
      while(rotate){
        wdt_reset();
        K1_Rotate_CCW_Switch.loop();
        if(K1_Rotate_CCW_Switch.isPressed()){
          Serial.print("K1 rotate CCW Limit detected\n");
          rotate = false;
        }
      }

      if(check_K1_Rotate_CCW_Limit() == false){
        Serial.print("K1 rotate CCW Limit confirmed\n");
        checkConfirm = false;
      }

      else {
        K1_Check = true;
        checkConfirm = true;
      }

      currentMillis = millis();
      if (currentMillis - startMillis >= watchdogTimeoutKR){
```

```
        while(1){}
      }
    }
    K1_rotate_STOP();
  }

  else {
    Serial.print("K1 Rotate is already at CCW limit\n");
  }
}

//Knight 2: Rotate Motor Handler Functions
void engage_K2_Rotate_CW(){
  //Check to make sure K2 can safely rotate CW then engage
  //Stop motor when limit has been detected

  Serial.print("K2 Rotate called for CW\n");
   K2_Check = true;
   checkConfirm = true;

  //Check K2 Rotate CW Limit Switch
  if(check_K2_Rotate_CW_Limit()){
    Serial.print("Safe to engage K2 Rotate CW\n");
    K2_rotate_CW();
    startMillis = millis();

    while(checkConfirm){
      wdt_reset();
      while(rotate){
        wdt_reset();
        K2_Rotate_CW_Switch.loop();
        if(K2_Rotate_CW_Switch.isPressed()){
          Serial.print("K2 rotate CW Limit detected\n");
          rotate = false;
        }
      }

      if(check_K2_Rotate_CW_Limit() == false){
        Serial.print("K2 rotate CW Limit confirmed\n");
        checkConfirm = false;
      }

      else {
        K2_Check = true;
```

```
      checkConfirm = true;
    }

    currentMillis = millis();
    if (currentMillis - startMillis >= watchdogTimeoutKR){
      while(1){}
    }
   }
   K2_rotate_STOP();
  }

  else {
   Serial.print("K2 Rotate is already at CW limit\n");
  }
}
void engage_K2_Rotate_CCW(){
  //Check to make sure K2 can safely rotate CCW then engage
  //Stop motor when limit has been detected

  Serial.print("K2 Rotate called for CCW\n");
   K2_Check = true;
   checkConfirm = true;

  //Check K2 Rotate CCW Limit Switch
  if(check_K2_Rotate_CCW_Limit()){
   Serial.print("Safe to engage K2 Rotate CCW\n");
   K2_rotate_CCW();
   startMillis = millis();

   while(checkConfirm){
    wdt_reset();
    while(rotate){
      wdt_reset();
      K2_Rotate_CCW_Switch.loop();
      if(K2_Rotate_CCW_Switch.isPressed()){
        Serial.print("K2 rotate CCW Limit detected\n");
        rotate = false;
      }
    }

    if(check_K2_Rotate_CCW_Limit() == false){
      Serial.print("K2 rotate CCW Limit confirmed\n");
      checkConfirm = false;
    }
```

```
      else {
        K2_Check = true;
        checkConfirm = true;
      }

      currentMillis = millis();
      if (currentMillis - startMillis >= watchdogTimeoutKR){
        while(1){}
      }
    }
    K2_rotate_STOP();
  }

  else {
    Serial.print("K2 Rotate is already at CCW limit\n");
  }
}

//Both Lances Motor Handler Functions
void lowerLances(){
  //Check to make sure K1 and K2 can safely lower Lances
  //Stop motor when limit has been detected

  Serial.print("Call for: Both Lances Down\n");

  K1_Check      = false;
  K2_Check      = false;
  K1_Check_2    = false;
  K2_Check_2    = false;
  checkConfirm  = false;
  startMillis   = millis();

  //Check  K1_LanceDown_Limit Switch
  if (check_K1_LanceDown_Limit()){
    //Serial.print("Lances Down MSG: 1\n");
    delay(joustDelay);
    K1_LanceDown();
  }
  if (!check_K1_LanceDown_Limit()) {
    //Serial.print("Lances Down MSG: 2\n");
    delay(joustDelay);
    Serial.print("K1 Lance already DOWN\n");
    K1_Check = true;
```

```
}

if (check_K2_LanceDown_Limit()){
  //Serial.print("Lances Down MSG: 3\n");
  delay(joustDelay);
  K2_LanceDown();
}
if (!check_K2_LanceDown_Limit()){
  //Serial.print("Lances Down MSG: 4\n");
  delay(joustDelay);
  Serial.print("K2 Lance already DOWN\n");
  K2_Check = true;
}

if ( K1_Check && K2_Check){
  //Serial.print("Lances Down MSG: 5\n");
  delay(joustDelay);
  checkConfirm = true;
}

while((!K1_Check) || (!K2_Check)){
  wdt_reset();
  //Serial.print("Lances Down MSG: 6\n");
  delay(joustDelay);
  K1_LanceDown_Switch.loop();
  if((K1_LanceDown_Switch.isPressed()) && (!K1_Check)){
    //Serial.print("Lances Down MSG: 7\n");
    delay(joustDelay);
    K1_LanceDown_Switch.loop();
    Serial.print("K1 Lance Down Limit detected\n");
    K1_lance_STOP();
    K1_Check = true;
  }

  //Serial.print("Lances Down MSG: 8\n");
  delay(joustDelay);

  K2_LanceDown_Switch.loop();
  if((K2_LanceDown_Switch.isPressed()) && (!K2_Check)){
    //Serial.print("Lances Down MSG: 9\n");
    delay(joustDelay);
    K2_LanceDown_Switch.loop();
    Serial.print("K2 Lance Down Limit detected\n");
    K2_lance_STOP();
```

```
      K2_Check = true;
    }

    currentMillis = millis();
    if (currentMillis - startMillis >= lanceMillisPeriod){
      Serial.print("Lance Limit Error Manual Override");
      startMillis = currentMillis;
      lanceFail++;
      K1_Check = true;
      K2_Check = true;
    }

    currentMillis = millis();
    if (currentMillis - startMillis >= watchdogTimeoutL){
      while(1){}
    }

    //Serial.print("Lances Down MSG: 10\n");
    delay(joustDelay);
  }
  //Serial.print("Lances Down MSG: 11\n");
  delay(joustDelay);
}

void raiseLances(){
  //Check to make sure K1 and K2 can safely raise Lances
  //Stop motor when limit has been detected

  Serial.print("Call for: Both Lances Up\n");

  K1_Check    = false;
  K2_Check    = false;
  K1_Check_2  = false;
  K2_Check_2  = false;
  checkConfirm     = false;
  startMillis = millis();

  //Check  K1_LanceUp_Limit Switch
  if(check_K1_LanceUp_Limit()){
    //Serial.print("Lances UP MSG: 1\n");
    delay(joustDelay);
    K1_LanceUp();
  }
  if(!check_K1_LanceUp_Limit()){
```

```
  //Serial.print("Lances UP MSG: 2\n");
  delay(joustDelay);
  Serial.print("K1 Lance already UP\n");
  K1_Check = true;
}

if (check_K2_LanceUp_Limit()){
  //Serial.print("Lances UP MSG: 3\n");
  delay(joustDelay);
  K2_LanceUp();
}
if (!check_K2_LanceUp_Limit()){
  //Serial.print("Lances UP MSG: 4\n");
  delay(joustDelay);
  K2_Check = true;
  Serial.print("K2 Lance already UP\n");
}

if ( K1_Check && K2_Check){
  //Serial.print("Lances UP MSG: 5\n");
  delay(joustDelay);
  checkConfirm = true;
}

while((!K1_Check) || (!K2_Check)){
  wdt_reset();
  //Serial.print("Lances UP MSG: 6\n");
  delay(joustDelay);
  K1_LanceUp_Switch.loop();
  if((K1_LanceUp_Switch.isPressed()) && (!K1_Check)){
    //Serial.print("Lances UP MSG: 7\n");
    delay(joustDelay);
    K1_LanceUp_Switch.loop();
    Serial.print("K1 Lance UP Limit detected\n");
    K1_lance_STOP();
    K1_Check = true;
  }

  //Serial.print("Lances UP MSG: 8\n");
  delay(joustDelay);

  K2_LanceUp_Switch.loop();
  if((K2_LanceUp_Switch.isPressed()) && (!K2_Check)){
    //Serial.print("Lances UP MSG: 9\n");
```

```
      delay(joustDelay);
      K2_LanceUp_Switch.loop();
      Serial.print("K2 Lance UP Limit detected\n");
      K2_lance_STOP();
      K2_Check = true;
    }

    //Serial.print("Lances UP MSG: 10\n");
    delay(joustDelay);

    currentMillis = millis();
    if (currentMillis - startMillis >= lanceMillisPeriod){
      Serial.print("Lance Limit Error Manual Override");
      startMillis = currentMillis;
      lanceFail++;
      K1_Check = true;
      K2_Check = true;
    }

    if (currentMillis - startMillis >= watchdogTimeoutL){
      while(1){}
    }
  }
  //Serial.print("Lances UP MSG: 11\n");
  delay(joustDelay);
}

//Knight 1: Lance Motor Handler Functions
void engage_K1_LanceDown(){
  //Check to make sure K1 can safely Lower Lance
  //Stop motor when limit has been detected

  Serial.print("Call for: K1 LanceDown\n");
  K1_Check      = false;
  checkConfirm  = false;
  startMillis = millis();

  if (check_K1_LanceDown_Limit()){
    delay(joustDelay);
    K1_LanceDown();
  }
  if (!check_K1_LanceDown_Limit()) {
    delay(joustDelay);
    Serial.print("K1 Lance already DOWN\n");
```

```
    K1_Check = true;
  }

  if (K1_Check){
    delay(joustDelay);
    checkConfirm = true;
  }

  while(!K1_Check){
    wdt_reset();
    delay(joustDelay);
    K1_LanceDown_Switch.loop();
    if((K1_LanceDown_Switch.isPressed()) && (!K1_Check)){
      delay(joustDelay);
      K1_LanceDown_Switch.loop();
      Serial.print("K1 Lance Down Limit detected\n");
      K1_lance_STOP();
      K1_Check = true;
    }

    currentMillis = millis();
    if (currentMillis - startMillis >= lanceMillisPeriod){
      Serial.print("Lance Limit Error Manual Override");
      startMillis = currentMillis;
      lanceFail++;
      K1_Check = true;
    }

    if (currentMillis - startMillis >= watchdogTimeoutL){
      while(1){}
    }
  }
}
void engage_K1_LanceUp(){
  //Check to make sure K1 can safely Raise Lance
  //Stop motor when limit has been detected

  Serial.print("Call for: K1 Lance Up\n");
  K1_Check      = false;
  checkConfirm  = false;
  startMillis = millis();

  //Check  K1_LanceUp_Limit Switch
  if(check_K1_LanceUp_Limit()){
```

```
    delay(joustDelay);
    K1_LanceUp();
  }
  if(!check_K1_LanceUp_Limit()){
    delay(joustDelay);
    Serial.print("K1 Lance already UP\n");
    K1_Check = true;
  }

  if (K1_Check){
    delay(joustDelay);
    checkConfirm = true;
  }

  while(!K1_Check){
    wdt_reset();
    delay(joustDelay);
    K1_LanceUp_Switch.loop();
    if((K1_LanceUp_Switch.isPressed()) && (!K1_Check)){
      delay(joustDelay);
      K1_LanceUp_Switch.loop();
      Serial.print("K1 Lance UP Limit detected\n");
      K1_lance_STOP();
      K1_Check = true;
    }

    currentMillis = millis();
    if (currentMillis - startMillis >= lanceMillisPeriod){
      Serial.print("Lance Limit Error Manual Override");
      startMillis = currentMillis;
      lanceFail++;
      K1_Check = true;
    }

    if (currentMillis - startMillis >= watchdogTimeoutL){
      while(1){}
    }
  }
}

//Knight 2: Lance Motor K2 Handler Functions
void engage_K2_LanceDown(){
  //Check to make sure K2 can safely Lower Lance
  //Stop motor when limit has been detected
```

```
Serial.print("Call for: K2 LanceDown\n");
K2_Check      = false;
checkConfirm  = false;
startMillis = millis();

if (check_K2_LanceDown_Limit()){
  delay(joustDelay);
  K2_LanceDown();
}
if (!check_K2_LanceDown_Limit()) {
  delay(joustDelay);
  Serial.print("K2 Lance already DOWN\n");
  K2_Check = true;
}

if (K2_Check){
  delay(joustDelay);
  checkConfirm = true;
}

while(!K2_Check){
  wdt_reset();
  delay(joustDelay);
  K2_LanceDown_Switch.loop();
  if((K2_LanceDown_Switch.isPressed()) && (!K2_Check)){
    delay(joustDelay);
    K2_LanceDown_Switch.loop();
    Serial.print("K2 Lance Down Limit detected\n");
    K2_lance_STOP();
    K2_Check = true;
  }

  currentMillis = millis();
  if (currentMillis - startMillis >= lanceMillisPeriod){
    Serial.print("Lance Limit Error Manual Override");
    startMillis = currentMillis;
    lanceFail++;
    K2_Check = true;
  }

  if (currentMillis - startMillis >= watchdogTimeoutL){
    while(1){}
  }
```

```
  }
}
void engage_K2_LanceUp(){
  //Check to make sure K2 can safely Raise Lance
  //Stop motor when limit has been detected

  Serial.print("Call for: K2 Lance Up\n");
  K2_Check      = false;
  checkConfirm  = false;
  startMillis = millis();

  if (check_K2_LanceUp_Limit()){
    delay(joustDelay);
    K2_LanceUp();
  }

  //Serial.print("K2 Lance MSG: 1\n");

  if (!check_K2_LanceUp_Limit()){
    //Serial.print("K2 Lance MSG: 2\n");
    delay(joustDelay);
    K2_Check = true;
    Serial.print("K2 Lance already UP\n");
  }

  if (K2_Check){
    //Serial.print("K2 Lance MSG: 3\n");
    delay(joustDelay);
    checkConfirm = true;
  }

  while(!K2_Check){
    wdt_reset();
    //Serial.print("K2 Lance MSG: 4\n");
    delay(joustDelay);
    K2_LanceUp_Switch.loop();
    if((K2_LanceUp_Switch.isPressed()) && (!K2_Check)){
      K2_LanceUp_Switch.loop();
      //Serial.print("K2 Lance MSG: 5\n");
      delay(joustDelay);
      Serial.print("K2 Lance UP Limit detected\n");
      K2_lance_STOP();
      K2_Check = true;
    }
```

```
    currentMillis = millis();
    if (currentMillis - startMillis >= lanceMillisPeriod){
      Serial.print("Lance Limit Error Manual Override");
      startMillis = currentMillis;
      lanceFail++;
      K1_Check = true;
      K2_Check = true;
    }

    if (currentMillis - startMillis >= watchdogTimeoutL){
      while(1){}
    }
  }
}

/***** Limit Switch Checks *****/
//Check the state of the limit switch. Value of TRUE means the limit switch is OPEN and motor
can be engaged.

//Check that knights are in a safe starting position
bool check_Safe_Operation(){
  if (((check_K1_Rotate_CW_Limit()) && (check_K1_Rotate_CCW_Limit())) &&
((check_K2_Rotate_CW_Limit()) && (check_K2_Rotate_CCW_Limit()))){
    if (!check_ChainDrive_CW_Limit() || !check_ChainDrive_CW_Limit()){
      Serial.print("Safe to start\n");
      return true;
    }
    else {
      Serial.print("Not Safe to start\n");
      return false;
    }
  }

  else{
    Serial.print("Safe to start\n");
    return true;
  }
}

//Check the start button
bool check_startButton(){
  if(digitalRead(startButtonPin)){
    return true;
```

```
  }

  else{
    return false;
  }
}

//Lock
bool check_LOCK(){
  //Check the state of the lock limit switch. Value of TRUE means the limit switch is OPEN and
Joust can engage.
  if (digitalRead(UNLOCK)){
    return true;
  }

  else {
    return false;
  }
}

//Chain Drive Limits
bool check_ChainDrive_CW_Limit(){
  //Check the state of the ChainDrive_CW_Limit switch. Value of TRUE means the limit switch is
OPEN and motor can be engaged.
  if (digitalRead(ChainDrive_CW_Limit)){
    return true;
  }

  else {
    return false;
  }
}
bool check_ChainDrive_CCW_Limit(){
  //Check the state of the ChainDrive_CCW_Limit switch. Value of TRUE means the limit switch
is OPEN and motor can be engaged.
  if (digitalRead(ChainDrive_CCW_Limit)){
    return true;
  }

  else {
    return false;
  }
}
```

```
//Rotate - K1
bool check_K1_Rotate_CW_Limit(){
  //Check the state of the K1_Rotate_CW_Limit switch. Value of TRUE means the limit switch is
OPEN and motor can be engaged.
  if (digitalRead(K1_Rotate_CW_Limit)){
    return true;
  }

  else {
    return false;
  }
}
bool check_K1_Rotate_CCW_Limit(){
  //Check the state of the K1_Rotate_CCW_Limit switch. Value of TRUE means the limit switch
is OPEN and motor can be engaged.
  if (digitalRead(K1_Rotate_CCW_Limit)){
    return true;
  }

  else {
    return false;
  }
}

//Rotate - K2
bool check_K2_Rotate_CW_Limit(){
  //Check the state of the K2_Rotate_CW_Limit switch. Value of TRUE means the limit switch is
OPEN and motor can be engaged.
  if (digitalRead(K2_Rotate_CW_Limit)){
    return true;
  }

  else {
    return false;
  }
}
bool check_K2_Rotate_CCW_Limit(){
  //Check the state of the K2_Rotate_CCW_Limit switch. Value of TRUE means the limit switch
is OPEN and motor can be engaged.
  if (digitalRead(K2_Rotate_CCW_Limit)){
    return true;
  }

  else {
```

```
    return false;
  }
}

//Lance - K1
bool check_K1_LanceDown_Limit(){
  //Check the state of the K1_LanceDown_Limit switch. Value of TRUE means the limit switch is
OPEN and motor can be engaged.
  if (digitalRead(K1_LanceDown_Limit)){
    return true;
  }

  else {
    return false;
  }
}
bool check_K1_LanceUp_Limit(){
  //Check the state of the K1_LanceUp_Limit switch. Value of TRUE means the limit switch is
OPEN and motor can be engaged.
  if (digitalRead(K1_LanceUp_Limit)){
    return true;
  }

  else {
    return false;
  }
}

//Lance - K2
bool check_K2_LanceDown_Limit(){
  //Check the state of the K2_LanceDown_Limit switch. Value of TRUE means the limit switch is
OPEN and motor can be engaged.
  if (digitalRead(K2_LanceDown_Limit)){
    return true;
  }

  else {
    return false;
  }
}
bool check_K2_LanceUp_Limit(){
  //Check the state of the K2_LanceUp_Limit switch. Value of TRUE means the limit switch is
OPEN and motor can be engaged.
  if (digitalRead(K2_LanceUp_Limit)){
```

```
    return true;
  }

  else {
    return false;
  }
}

bool check_Both_LancesUp_Button(){
  //Check the state of the BOTH_LANCES_UP button. Value of TRUE means the button has
been pressed.
  if (digitalRead(BOTH_LANCES_UP)){
    return true;
  }

  else {
    return false;
  }
}


/***** Motor Control Functions *****/

//Chain Drive Motor
void chainDrive_CW(){
  //Clock Wise Rotate - Chain Drive Motor - [CW]
  //Direct control of chain drive to send Knight 1 to the [LEFT], Knight 2 to the [RIGHT]
  Serial.print("Chain Drive: CW\n");
  digitalWrite(MOTOR_RELAY_CW,  LOW );
  digitalWrite(MOTOR_RELAY_CCW, HIGH);
}
void chainDrive_CCW(){
  //Counter Clock Wise Rotate - Chain Drive Motor - [CCW]
  //Direct control of chain drive to send Knight 1 to the [RIGHT], Knight 2 to the [LEFT]
  Serial.print("Chain Drive: CCW\n");
  digitalWrite(MOTOR_RELAY_CW,  HIGH);
  digitalWrite(MOTOR_RELAY_CCW, LOW );
}
void chainDrive_STOP(){
  //Stop rotate of Chain Drive Motor
  Serial.print("Chain Drive: STOP\n");
  digitalWrite(MOTOR_RELAY_CW,  HIGH);
  digitalWrite(MOTOR_RELAY_CCW, HIGH);
}
```

```
//Rotate Motor - K1
void K1_rotate_CW(){
  //Knight 1 Rotates CW
  Serial.print("K1 Rotate: CW\n");
  M2_K1_ROTATE->setSpeed(rotateSpeed);
  M2_K1_ROTATE->run(BACKWARD);
}
void K1_rotate_CCW(){
  //Knight 1 Rotates CCW
  Serial.print("K1 Rotate: CCW\n");
  M2_K1_ROTATE->setSpeed(rotateSpeed);
  M2_K1_ROTATE->run(FORWARD);
}
void K1_rotate_STOP(){
  //Knight 1 Stops Rotate
  Serial.print("K1 Rotate: STOP\n");
  M2_K1_ROTATE->run(RELEASE);
}

//Rotate Motor - K2
void K2_rotate_CW(){
  //Knight 2 Rotates CW
  Serial.print("K2 Rotate: CW\n");
  M4_K2_ROTATE->setSpeed(rotateSpeed);
  M4_K2_ROTATE->run(BACKWARD);
}
void K2_rotate_CCW(){
  //Knight 2 Rotates CCW
  Serial.print("K2 Rotate: CCW\n");
  M4_K2_ROTATE->setSpeed(rotateSpeed);
  M4_K2_ROTATE->run(FORWARD);
  Serial.print("K2 Rotate: CCW\n");

}
void K2_rotate_STOP(){
  //Knight 2 Stops Rotate
  Serial.print("K2 Rotate: STOP\n");
  M4_K2_ROTATE->run(RELEASE);
}

//Lance Motor - K1
void K1_LanceDown(){   //CW
  //Lower K1 Lance
```

```
  Serial.print("Lower K1 lance!\n");
  M1_K1_LANCE->setSpeed(lanceSpeed);
  M1_K1_LANCE->run(FORWARD);
}
void K1_LanceUp(){  //CCW
  //Raise K1 Lance
  Serial.print("Raise K1 lance!\n");
  M1_K1_LANCE->setSpeed(lanceSpeed);
  M1_K1_LANCE->run(BACKWARD);
}
void K1_lance_STOP(){
  Serial.print("K1 Lance STOP\n");
  M1_K1_LANCE->run(RELEASE);
}

//Lance Motor - K2
void K2_LanceDown(){  //CW
  //Lower K2 Lance
  Serial.print("Lower K2 lance!\n");
  M3_K2_LANCE->setSpeed(lanceSpeed);
  M3_K2_LANCE->run(FORWARD);
}
void K2_LanceUp(){  //CCW
  //Raise K2 Lance
  Serial.print("Raise K2 lance!\n");
  M3_K2_LANCE->setSpeed(lanceSpeed);
  M3_K2_LANCE->run(BACKWARD);
}
void K2_lance_STOP(){
  Serial.print("K2 Lance STOP\n");
  M3_K2_LANCE->run(RELEASE);
}

//***** Button Loop Initializations *****/

void allButtonLoops(){
  //Limit Switches
  K1_LanceUp_Switch.loop();
  K2_LanceUp_Switch.loop();
  ChainDrive_CCW_Switch.loop();
  ChainDrive_CW_Switch.loop();
  K1_LanceDown_Switch.loop();
  K2_LanceDown_Switch.loop();
  K1_Rotate_CCW_Switch.loop();
```

```
  K2_Rotate_CCW_Switch.loop();
  K1_Rotate_CW_Switch.loop();
  K2_Rotate_CW_Switch.loop();

  //User Input Buttons
  startButton.loop();
  Both_Knights_Rotate_CW_Button.loop();
  Both_Knights_Rotate_CCW_Button.loop();
  Both_LancesDown_Button.loop();
  Both_LancesUp_Button.loop();
  ChainDrive_CW.loop();
  ChainDrive_CCW.loop();
  Unlock.loop();
}

void limitSwitchLoops(){
  //Limit Switches
  K1_LanceUp_Switch.loop();
  K2_LanceUp_Switch.loop();
  ChainDrive_CCW_Switch.loop();
  ChainDrive_CW_Switch.loop();
  K1_LanceDown_Switch.loop();
  K2_LanceDown_Switch.loop();
  K1_Rotate_CCW_Switch.loop();
  K2_Rotate_CCW_Switch.loop();
  K1_Rotate_CW_Switch.loop();
  K2_Rotate_CW_Switch.loop();
}

void userInputLoops(){
  //User Input Buttons
  startButton.loop();
  Both_Knights_Rotate_CW_Button.loop();
  Both_Knights_Rotate_CCW_Button.loop();
  Both_LancesDown_Button.loop();
  Both_LancesUp_Button.loop();
  ChainDrive_CW.loop();
  ChainDrive_CCW.loop();
  Unlock.loop();
}
```